Divide and Conquer
Randomized Algorithms

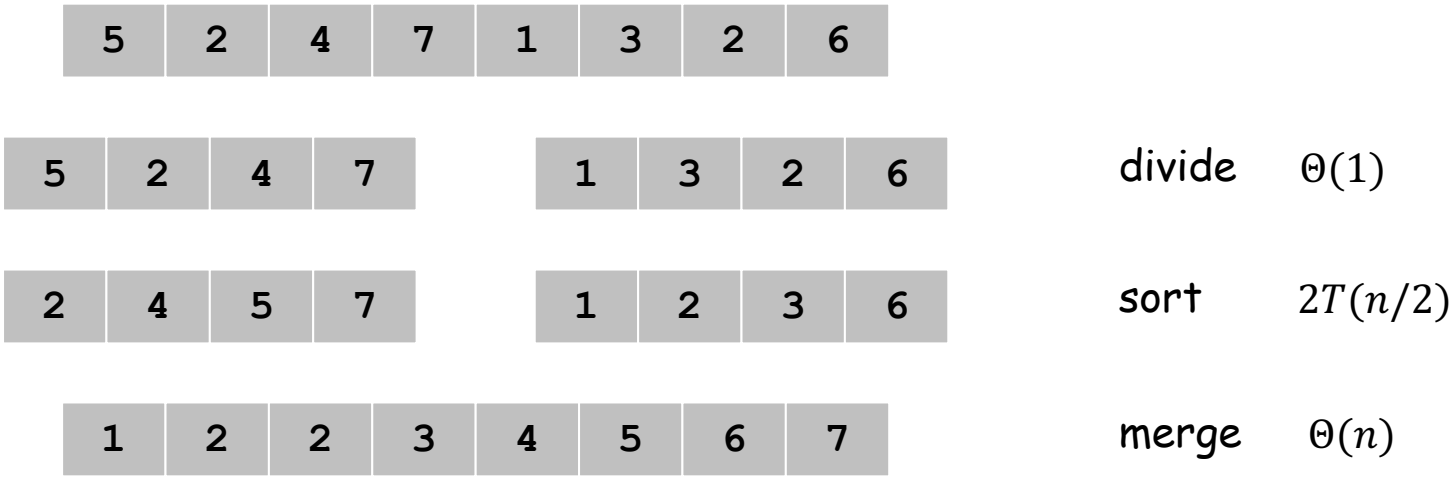
Outline

1. Quicksort & Partition
2. Randomization
3. Analysis of Randomized Quicksort
4. Randomized Selection

Quicksort: The "dual" of merge sort

Mergesort(A, p, r):
if $p = r$ **then return**
$q \leftarrow \lfloor (p + r)/2 \rfloor$
Mergesort(A, p, q)
Mergesort$(A, q + 1, r)$
Merge(A, p, q, r)

First call: **Mergesort**$(A, 1, n)$



Quicksort: The "dual" of merge sort [II]

Mergesort(A, p, r)**:**

if $p = r$ **then return**

$q \leftarrow \lfloor (p + r)/2 \rfloor$

Mergesort(A, p, q)

Mergesort$(A, q + 1, r)$

Merge(A, p, q, r)

First call: **Mergesort**$(A, 1, n)$

Quicksort(A, p, r)**:**

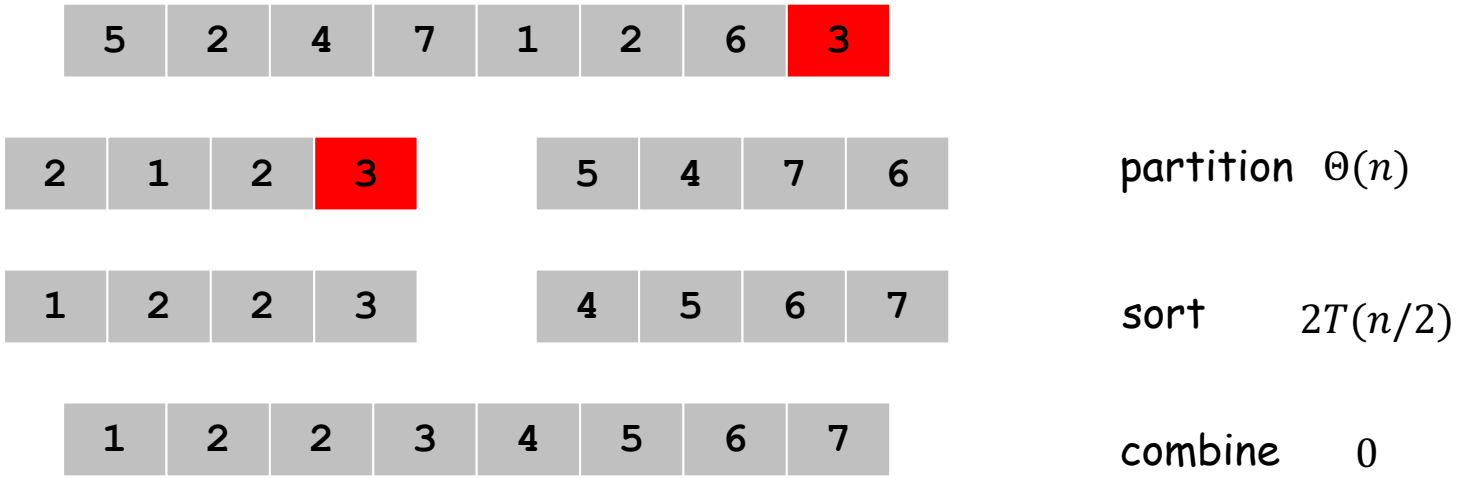
if $p \geq r$ **then return**

$q =$ **Partition**(A, p, r)

Quicksort$(A, p, q - 1)$

Quicksort$(A, q + 1, r)$

First call: **Quicksort**$(A, 1, n)$



Quicksort: The "dual" of merge sort [III]

Quicksort chooses an item as pivot.

It *partitions* array so that all items less than or equal to pivot are on the left and all items greater than pivot on the right.

It then recursively Quicksorts left and right sides.

Quicksort(A, p, r)**:**

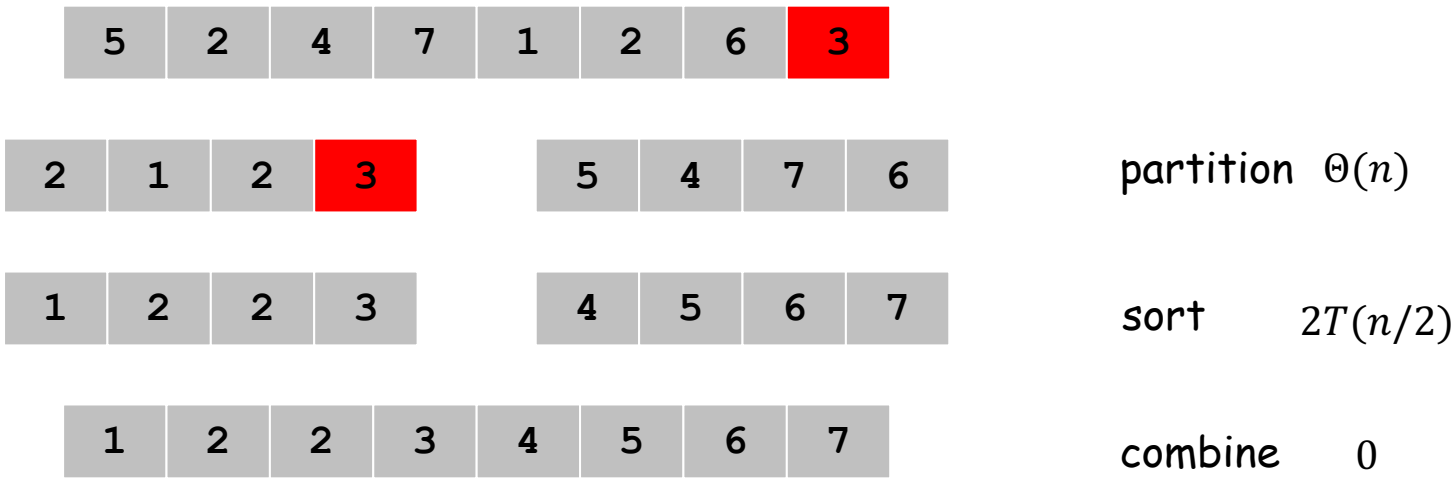
if $p \geq r$ **then return**

$q =$ **Partition**(A, p, r) **//new pivot position**

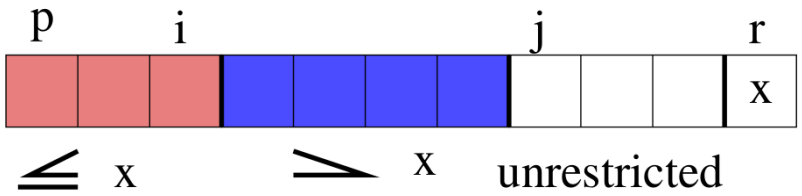
Quicksort$(A, p, q-1)$

Quicksort$(A, q+1, r)$

First call: **Quicksort**$(A, 1, n)$



Partition with the last element as the pivot



Partition(A, p, r) **:**

$x \leftarrow A[r]$

$i \leftarrow p - 1$

for $j \leftarrow p$ **to** $r - 1$

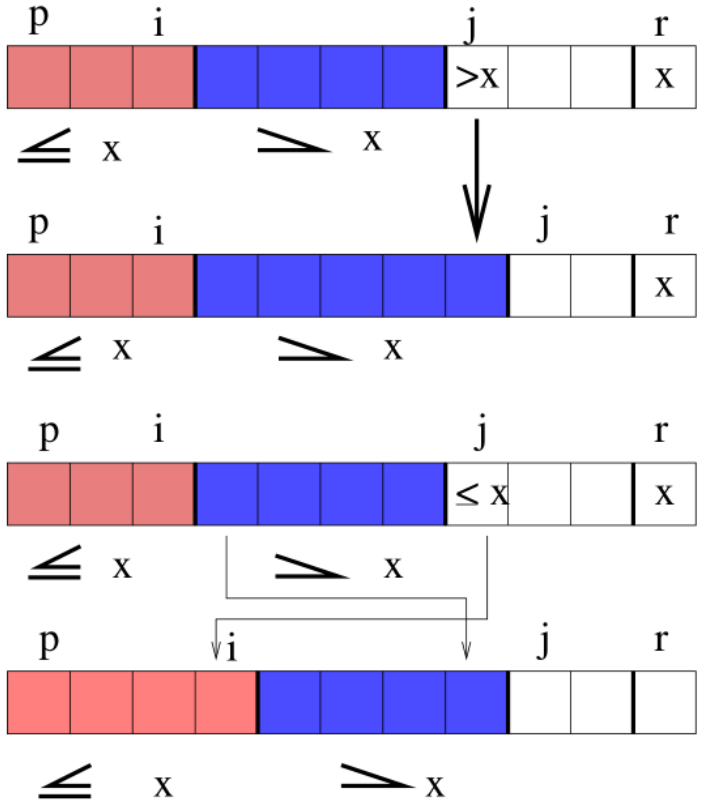
  if $A[j] \leq x$ **then**

    $i \leftarrow i + 1$

    swap $A[i]$ **and** $A[j]$

swap $A[i + 1]$ **and** $A[r]$

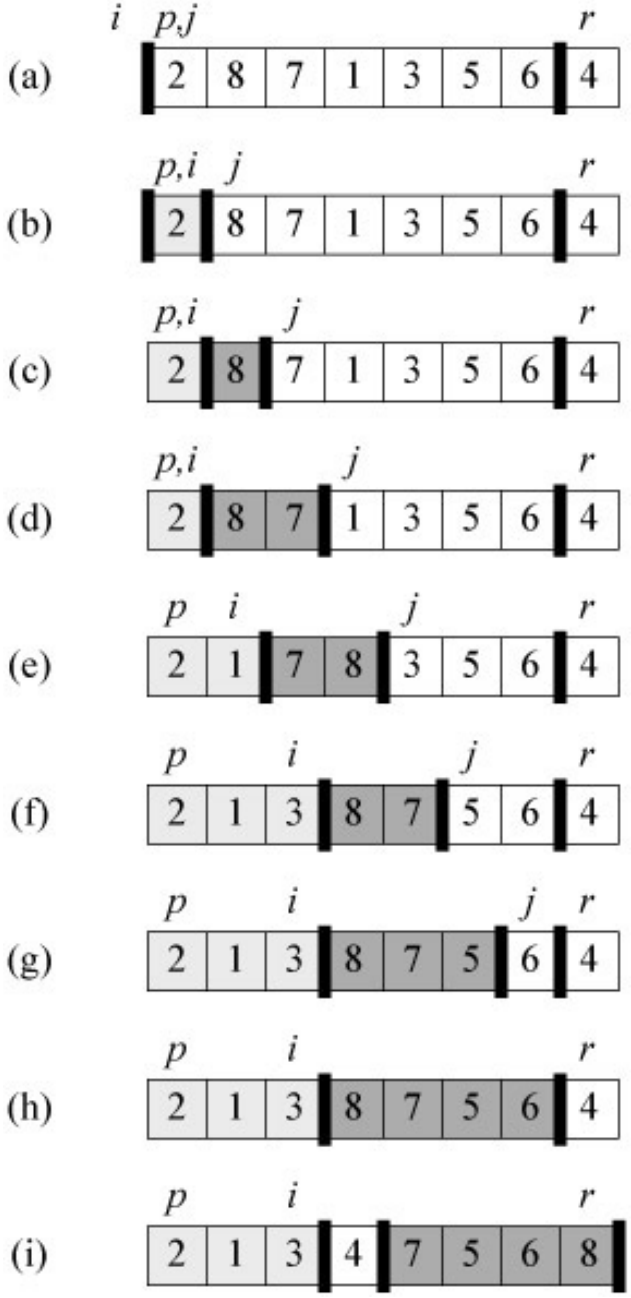
return $i + 1$ **//pivot position**



Time: $\Theta(n)$

Working space: $O(1)$ (in-place algorithm)

Partition: Example



Pivot selection is crucial

Running time.

- [Best case.] Select the median (middle value) element as the pivot: quicksort runs in $\Theta(n \log n)$ time *(problem is that we don't know the median).*
- [Worst case.] Select the smallest (or the largest) element as the pivot: quicksort runs in $\Theta(n^2)$ time.

Q: How to make running time **independent** of input

A: Randomly choose an element as the pivot
(swap it with last item in array and then run partition)

This is what's known as a ***randomized algorithm:***
algorithm makes a ***random*** choice each time it chooses a pivot.

It might help to think of the algorithm as trying to fool an evil adversary who is attempting to create a bad input by forcing bad pivots all the time. By making random pivot choices, the algorithm makes it impossible for the adversary to force a bad pivot, since the adversary can't know which key will be chosen as a pivot.

Analysis for Randomized Algorithms

Worst case almost never happens: Every pivot would have to be minimum or maximum; occurs with very low probability.

Expected running time of any input of size n.

Average case analysis	Expected case analysis
Used for deterministic algorithms	Used for randomized algorithms
Assume the input is chosen randomly from some distribution	Need to work for any input
Depends on assumptions on the input, weaker	Randomization is inherent within the algorithm, stronger

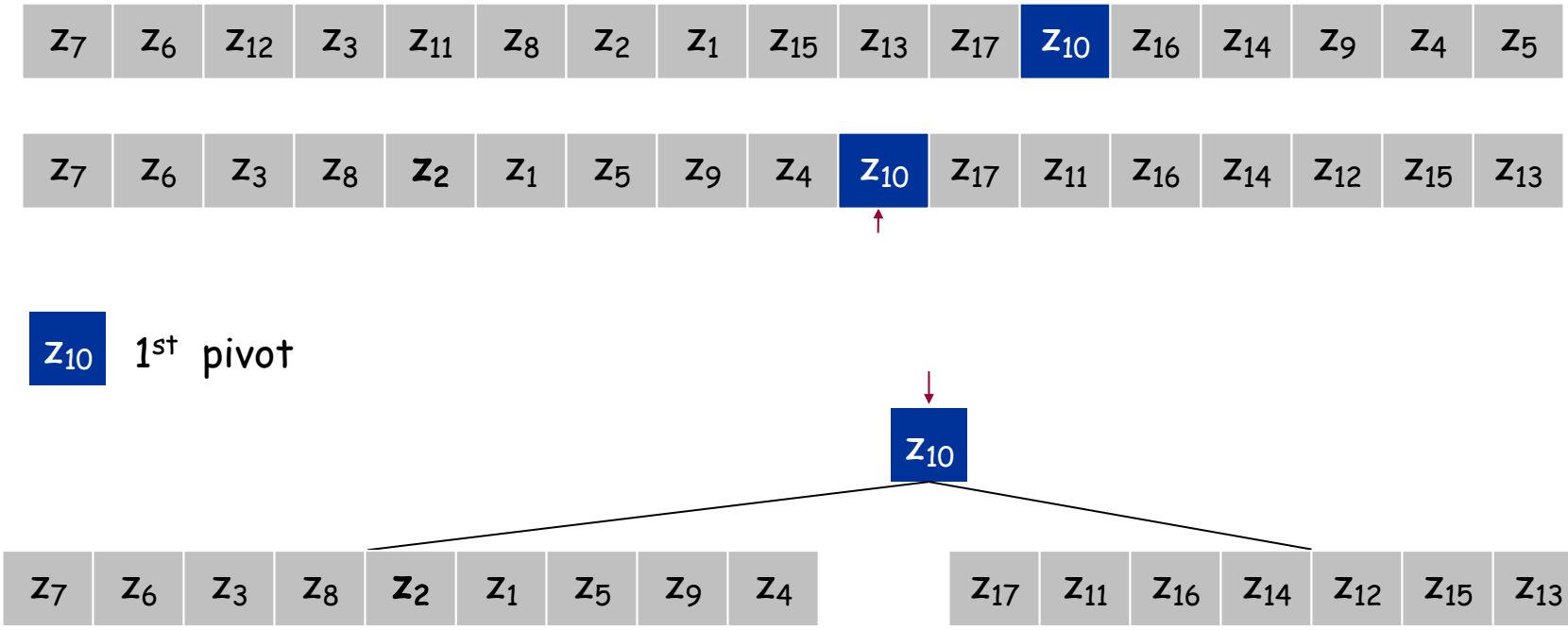


Our Randomized QuickSort Analysis

Analysis of Randomized Quicksort: The binary tree representation

Assumption: All elements are distinct - Relabel the elements from small to large as $z_1, z_2, \ldots, z_n$. We measure the running time based on the ***average number*** of comparisons performed.

Create a Binary tree corresponding to the Quicksort partitioning. The root of the tree will be the first pivot. All items to the left (right) of the root will be in its left (right) subtree.



Analysis of Randomized Quicksort: The binary tree representation

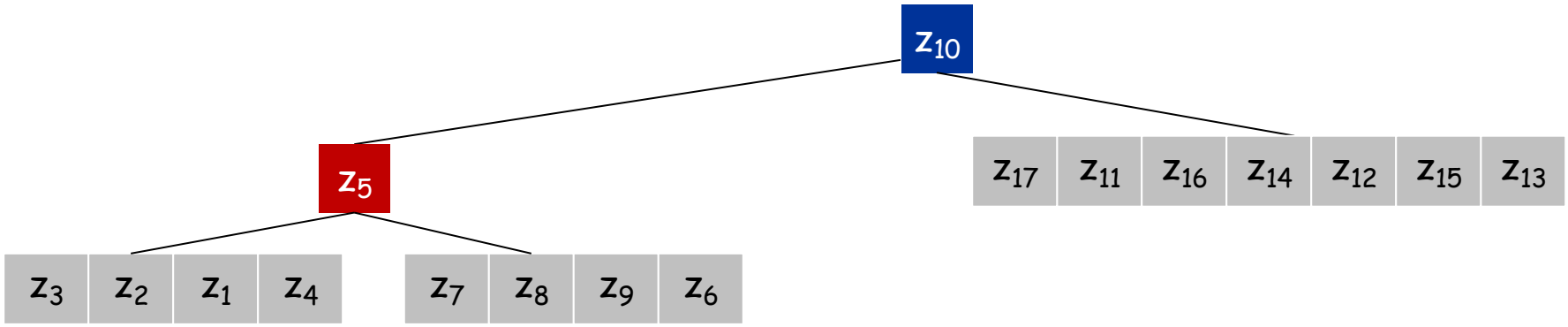
Create a Binary tree corresponding to the Quicksort partitioning

Pivots will be nodes: items in subarray to left of pivot will be in left subtree; items to right of pivot, in right subtree

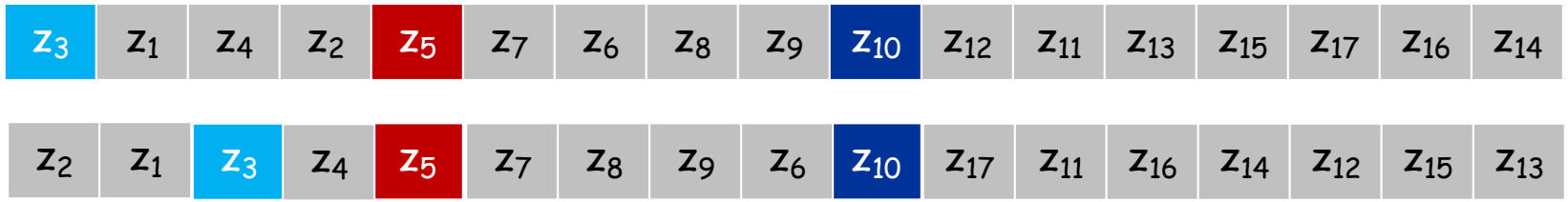
z_7	z_6	z_3	z_8	z_2	z_1	z_5	z_9	z_4	z_{10}	z_{17}	z_{11}	z_{16}	z_{14}	z_{12}	z_{15}	z_{13}

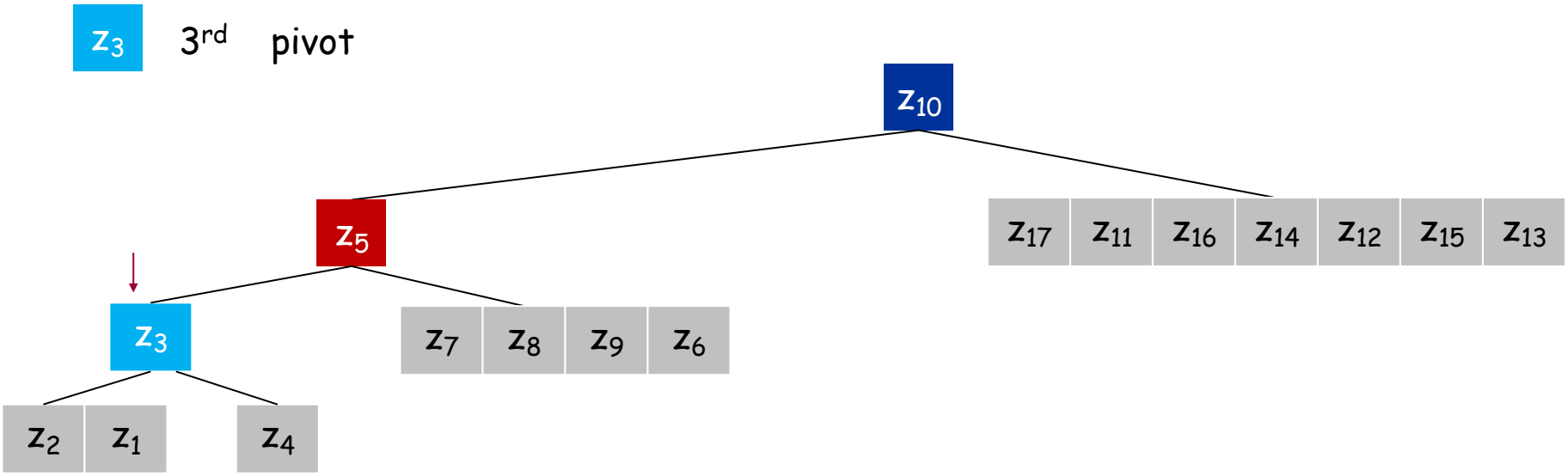
z_3	z_2	z_1	z_4	z_5	z_7	z_8	z_9	z_6	z_{10}	z_{12}	z_{11}	z_{13}	z_{15}	z_{17}	z_{16}	z_{14}

z_5 2nd pivot

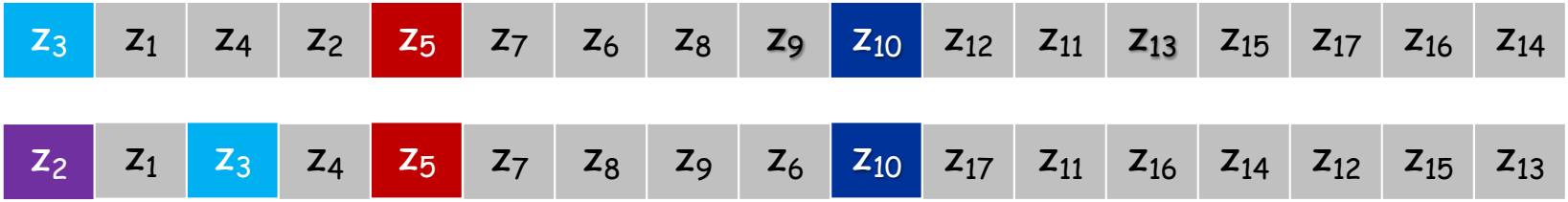


Analysis of Randomized Quicksort: The binary tree representation

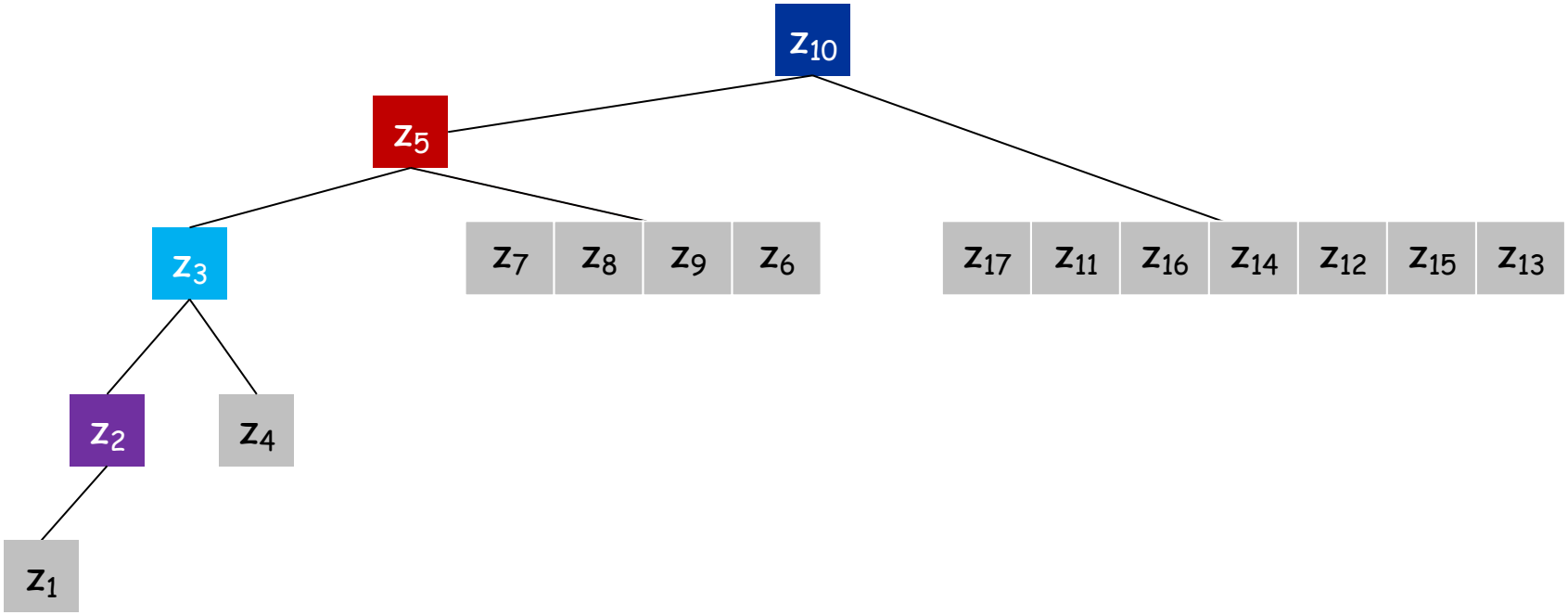




Analysis of Randomized Quicksort: The binary tree representation



z_2 4th pivot

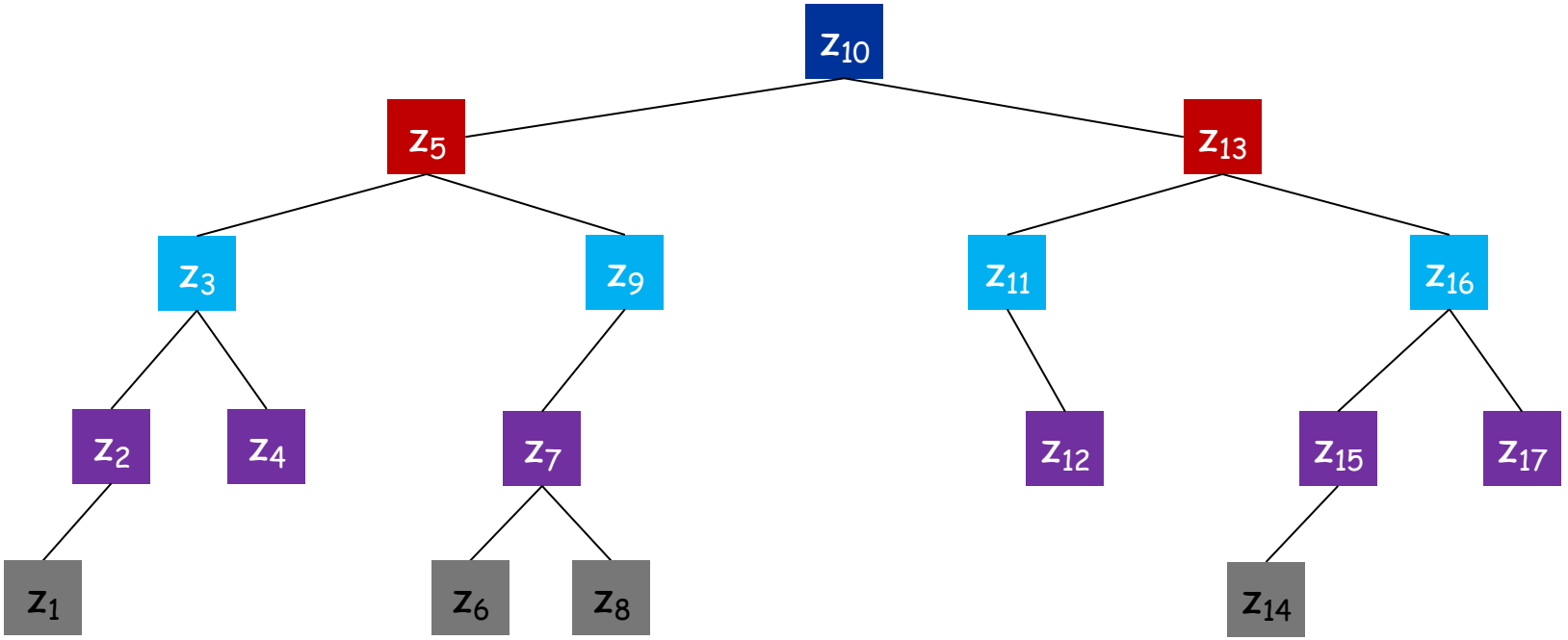


Analysis of quicksort: The binary tree representation

Keep going.

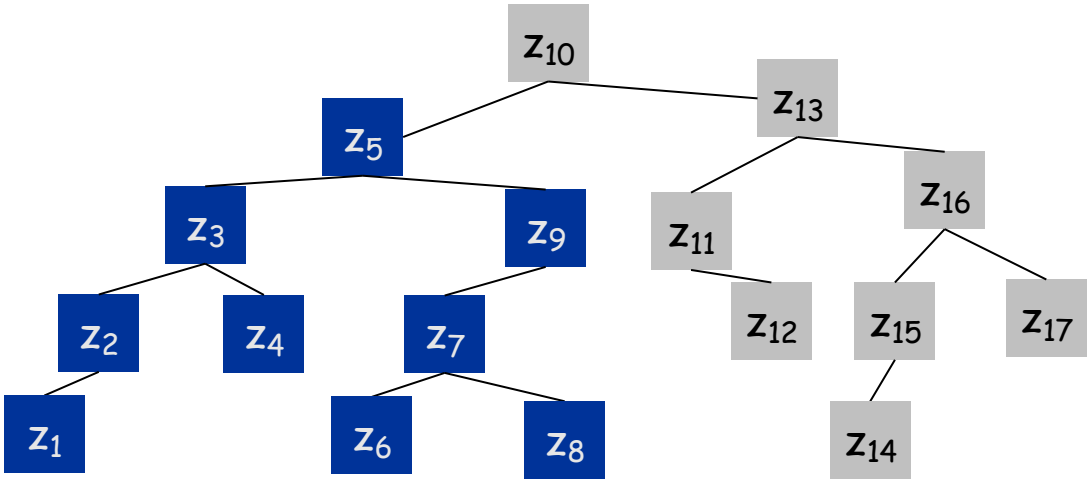
At the end we have a binary search tree that corresponds to the choice of pivots





Analysis of quicksort: The binary tree representation

z_1	z_2	z_3	z_4	z_5	z_6	z_7	z_8	z_9	z_{10}	z_{12}	z_{11}	z_{13}	z_{14}	z_{15}	z_{16}	z_{17}



The tree structure completely encodes the running of quicksort.

Let T_i be the subtree rooted at z_i

Ex: T_5 is the subtree rooted at z_5

This means that when z_i was a pivot, its subarray contained exactly the items in T_i.

Those items are then partitioned around z_i (corresponding to being placed in the left and right subtrees).

Fact:
(*) z_i is compared with z_j by Qsort
if and only if
(**) in the tree
z_i is an ancestor of z_j
or z_j is an ancestor of z_i

Summary of Observations

Elements z_i and z_j are compared once in Qsort iff in the binary tree:

1. z_i is an ancestor of z_j, i.e., z_i is the first pivot chosen among $\{z_i, z_{i+1}, \ldots, z_j\}$, or
2. z_j is an ancestor of z_i i.e., z_j is the first pivot chosen among $\{z_i, z_{i+1}, \ldots, z_j\}$

Otherwise, if another element $z_k \in \{z_i, z_{i+1}, \ldots, z_j\}$ is chosen as a pivot, z_i and z_j will go to different subtrees (subarrays) rooted at z_k and never be compared.

1. Define Indicator Random Variables: $X_{ij} = 1$ if z_i is compared with z_j
   Then # of comparisons is $X = \sum_{i<j} X_{ij}$
2. By Linearity of Expectation
   $E[\#\text{ of comparisons made by Qsort}] = \sum_{i<j} E[X_{ij}] = \sum_{i<j} \Pr[z_i, z_j \text{ compared}]$
3. Will show that
   $$\Pr[z_i, z_j \text{ compared}] = 2/(j-i+1)$$
4. And also show
   $E[\#\text{ of comparisons made by Qsort}] = \sum_{I<k} 2/(j-i+1) = O(n \log n).$

Analysis of quicksort

(*) z_i and z_j are compared iff z_i or z_j are chosen as the first pivot among $\{z_i, z_{i+1}, \ldots, z_j\}$

Consider the running of Quicksort.

1. For any pair i,j, consider first time that an item in $Z = \{z_i, z_{i+1}, \ldots, z_j\}$ is chosen as a pivot.
2. At each step of Qsort, every item in the current subarray is equally likely to be chosen as a pivot.
3. Because Qsort chooses pivots randomly, when the first pivot in Z is chosen it is equally likely to be any item in Z (which has size j-i+1).

$$\Rightarrow \Pr[z_i \text{ is first pivot chosen in } Z] = \frac{1}{j-i+1} = Pr[z_j \text{ is first pivot chosen in } Z]$$

$$\Rightarrow Pr[z_i \text{ and } z_j \text{ are compared}] = \Pr[\text{Either } z_i \text{ or } z_j \text{ are first pivot chosen in } Z] = \frac{2}{j-i+1}$$

4. E[# of comparisons made by Qsort] $= \sum_{i<j} E[X_{ij}] = \sum_{i<j} \Pr[\, z_i, z_j \text{ compared}] =$

$$\sum_{i<j} \frac{2}{j-i+1}$$

Analysis of quicksort (continued)

Theorem. Expected # of comparisons is $\Theta(n \log n)$.

Pf. Remains to show that $\sum_{i<j} \frac{2}{j-i+1} = \theta(n \log n)$

- Idea is to write out the summation as items in an upper triangular matrix and then show that each of the n rows has sum $O(\log n)$

	$j=2$	3	4	…	n	
$i=1$	$\frac{2}{2}$	$\frac{2}{3}$	$\frac{2}{4}$	…	$\frac{2}{n}$	$= O(\log n)$
2		$\frac{2}{2}$	$\frac{2}{3}$	…	$\frac{2}{n-1}$	$= O(\log n)$
3			$\frac{2}{2}$	…	$\frac{2}{n-2}$	$= O(\log n)$
…	…	…	…	…	…	…
$n-1$					$\frac{2}{2}$	$= O(\log n)$

2(1+1/2+…1/n)-1
1+1/2+…1/n is the harmonic series Θ(logn)

Since every row has sum $O(\log n)$ the entire matrix has sum $O(\text{n} \log n)$.